🗣️

MPI

Created	@February 24, 2025 8:45 PM
Class	High Performance Computing

- Message Passing Interface
- standard for distributed communication

- Basic outline:

```
# include <mpi.h>

int main(int argc, char* argv[]) {

    /* No MPI calls before this */
    MPI_Init(&argc, &argv);
    ...
    MPI_Finalize();
    /* No MPI calls after this */

    return 0;
```

Communicators

- sets of processes that an send messages to each other
- MPI_Init defines a communicator that consist of all the processes created when the program is started

- Called MPI_COMM_WORLD
- `MPI_Comm_size` gives the number of processes in the communicator
- `MPI_Comm_rank` gives the process number making the call

SPMD

- single program multiple data
- compile one program, each process does something different
- the if-else construct makes our program SPMD

- send and receive functions
- receiver waits for sender, matching tags
- communicator has to be the same

MPI_STATUS* status;

can tell you MPI_Source, MPI_TAG, and MPI_ERROR

MPI Collectives

Issues with send and receive:

- exact behavior is determined by the MPI implementation
- MPI_Send may behave differently with regard to buffer size, cutoffs, and blocking
- MPI_Recv always blocks until a matching message is received

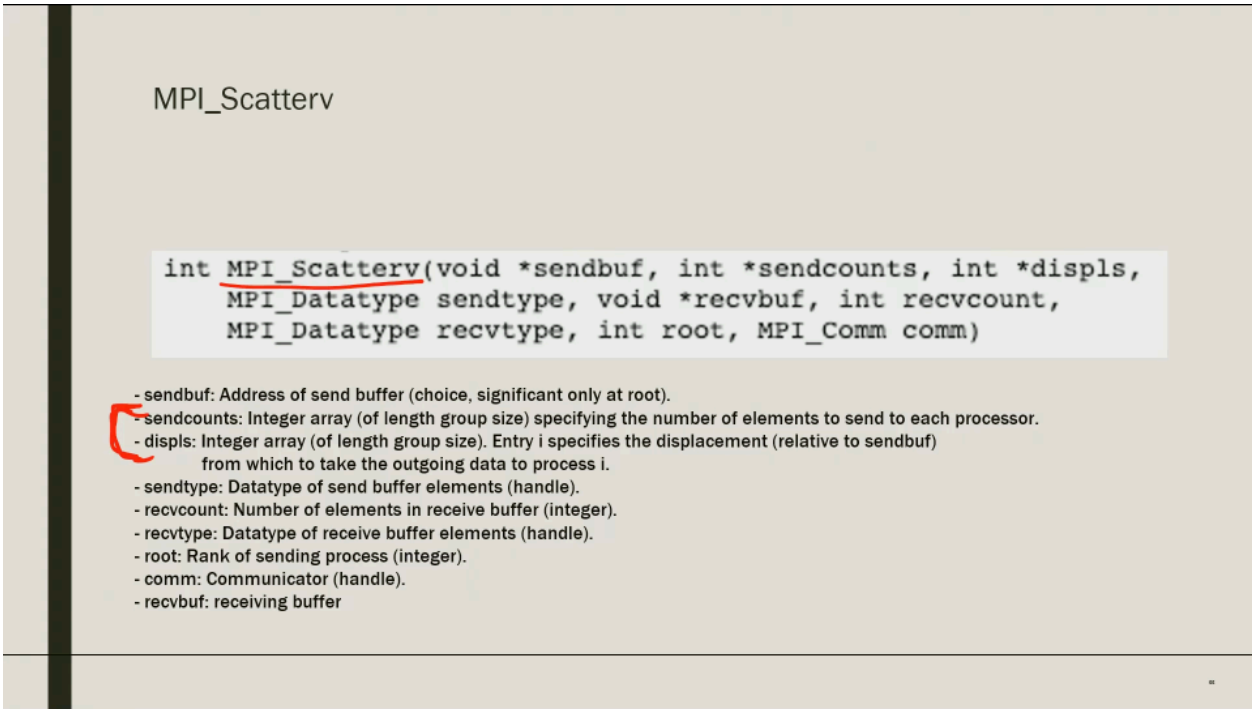
Managing Input of MPI Programs

- most MPI implementations only allow process 0 in MPI_COMM_WORLD access to stdin
- process 0 must read the data (scanf) and sent to the other processes

- all the process in the communicator must call the same collective function
- a program that attempts to match a call to MPI_Reduce on one processes with a call o MPI_Recv on another process s erroneous, and in all likelihood, the program will hang or crash
- output_data_p argument is only used on dest_process
- all the process need to pass in an argument corresponding to output_datA_p, even if its NULL
- point to point communications are matched on the basis of tags and communicators
- collective communications don’t use tags
- they’re matched solely on the basis of the communicator and the order in which they’re called

MPI_Allreduce

- useful in a situate in which all of the processes need the result of a global sum in order to complete some larger computation
-



- Gather: collect all of the components of the vector process on 0 and then 0 can use the vector, opposite of scatter

- Derived Datatypes:
  - Used to represent any collection of data items in memory by storing both the ty pees of the items and their relative locations in memory
  - the idea is that if a function that sends data knows this information about a collection of data items, it can collect the items from memory before they are sent
  - similarly, a function that receives data can distribute the items into their correct destinations in memory when they’re received
  - consists of a sequence of basic MPI data types together with a displacement for each of the data types

MPI_Type create_struct - builds a derived datatype that consists of individual elements that hae different basic types

MPI_Get_address - returns the address of the memory location referenced by location_p

- the special type MPI_Aint is an integer that is big enough to store an address on the system

MPI_Wtime returns the number of seconds that have elapsed since some time in the past

MPI_Barrier - ensures that noe process wil return from cal;ling it until every process in the communicator has started calling it

Safety in MPI Programs:

- The mpi standard allows MPI_Send to behave in two different ways
  - it can copy the message into an MPI managed buffer and return - async
  - or it can block until the matching call to MPI_Recv starts
  - many implementation of MPI set a threshold at which the system switches from buffering to blocking
  - relatively small messages will be buffered by MPI_Send
  - Larger messages will cause it to block
  - if the MPI_Send executed by each process blocks, no process will be able to start executing a call to MPI_Recv, and the program will hang or deadlock
  - each process is blocked waiting for an event that will never happen

- MPI Sendrecv
  - an alternative to scheduling the communications ourselves, carries out a blocking send and a receive in a single all
  - the dest and the source can be the same or different

    - especially useful because MPI schedules the communications so that the program wont hang or crash
    - point to point communications

- MPI_Ssend
    - synchronous send
    - guaranteed to block

For irregular connections:

- it is hard to figure out send receive sequence that does not deadlock or serialize
- even if you, you may have process idle time
- instead:
    - declare this data needs to be sent, these messages are expected and wait for them collectvely

Non-Blocking send/recv

- start non-blocking communication MPI_Isend, MPI_Irecv
- wait for Isend/Irecv calls to finish in any order: MPI_Wait();

Latency Hiding

- other motivation for non-blocking calls:
- overlap of computation and communications, provided hardware support
- known as latency hiding
- example:
    - start communication for edge points
    - do local operations while communication goes on,

    - wait for edge points from neighbor processes to incorporate incoming data

Parallel I/O

- multipe processs read from one file: no problem
- multiple writes to noe file: big problem
- everyone writes to separate file: stress on the file system, and requires post-processing

- MPI_File_open is a collective call to open a file
    - collective is defined on the communicator
    - typically use MPI_COMM_WORLD as communicator
    - to open a file just on one process use MPI_COMM_SELF as the communicator
- al processes must call MPI_FILE_OPEN with the same filename and amode parameters
- file handle fh is then used in all subsequent file operations
- amode defines access moe for the file, there are ultiple, and they can be combined
- MPI_File_close is a collective call to close a file
- all outstanding operations are synced on the file before it is closed

Displacements, elementary datatypes and offsets

- the displacement of a position within a file is the number of bytes form the beginning of the file

- etype ,or elementary datatype, is an MPI datatype
- an offset is a position in the file given in terms of multiples of etypes
- View, of the file is more natural, defined in terms of a displacemnt into the file and an etype, filetype, and data representation

MPI Examples

`MPI_Init` , `MPI_Comm_size` , `MPI_Comm_rank` , `MPI_Finalize`

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // Initialize the MPI execution environment.
    // This should be the very first MPI function called in any MPI program.
    MPI_Init(&argc, &argv);

    // Get the number of processes (also called ranks) in the communicator MPI_C
    // MPI_COMM_WORLD is the default communicator that includes all the proces
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Get the rank (unique identifier) of the calling process in the communicator.
    // Ranks are numbered from 0 to world_size - 1.
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Now that we know our rank and the number of processes, we can print a sin
    printf("Hello world from process %d of %d\n", world_rank, world_size);

    // Finalize the MPI environment.
```

```
    // This function cleans up the MPI environment and must be the last MPI call.
    MPI_Finalize();

    return 0;
}
```

`MPI_Send`, `MPI_Recv`

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char** argv) {
    // Initialize MPI environment.
    MPI_Init(&argc, &argv);

    // Get the rank and size within MPI_COMM_WORLD.
    int world_rank, world_size;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // We require exactly 2 processes for this example.
    if (world_size != 2) {
        if (world_rank == 0) {
            fprintf(stderr, "World size must be exactly two for this example\n");
        }
        MPI_Finalize();
        return 1;
    }

    // Define the message buffer.
    char message[20];
```

```
    if (world_rank == 0) {
        // Rank 0 prepares the message.
        strcpy(message, "Hello World");

        // Send the message to rank 1.
        // Parameters: buffer, count, datatype, destination rank, tag, communicator.
        MPI_Send(message, strlen(message) + 1, MPI_CHAR, 1, 0, MPI_COMM_WOR
        printf("Process 0 sent message: '%s'\n", message);
    }
    else if (world_rank == 1) {
        // Rank 1 receives the message from rank 0.
        // Parameters: buffer, count, datatype, source rank, tag, communicator, statu
        MPI_Recv(message, 20, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS
        printf("Process 1 received message: '%s'\n", message);
    }

    // Finalize the MPI environment.
    MPI_Finalize();

    return 0;
}
```

Multiple Tags in Send & Recv

```
// Process 0
MPI_Send(int_data, num_elements, MPI_INT, 1, 0, MPI_COMM_WORLD);   // Tag =
MPI_Send(str_data, strlen(str_data) + 1, MPI_CHAR, 1, 1, MPI_COMM_WORLD); //

// Process 1
MPI_Recv(int_data, num_elements, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STA
MPI_Recv(str_data, buffer_size, MPI_CHAR, 0, 1, MPI_COMM_WORLD, MPI_STATU
```

MPI_Reduce

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // 1) Initialize the MPI execution environment.
    //    Must be called before any other MPI function.
    MPI_Init(&argc, &argv);

    // 2) Query the total number of processes in MPI_COMM_WORLD.
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // 3) Query the rank (ID) of this process within MPI_COMM_WORLD.
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Each process prepares a value to contribute: here, its own rank.
    int local_value = world_rank;

    // This will hold the result of the reduction on the root process.
    int global_sum = 0;

    // 4) Perform the reduction across all processes.
    //    - sendbuf: address of local data          (&local_value)
    //    - recvbuf: address for result on root     (&global_sum)
    //    - count:   number of elements per process  (1)
    //    - datatype:MPI_INT for C `int`
    //    - op:      MPI_SUM to sum all local_values
    //    - root:    rank of the process that gets the result (0)
    //    - comm:    communicator (MPI_COMM_WORLD)
    MPI_Reduce(
        &local_value,   // sendbuf
        &global_sum,    // recvbuf (only significant at root)
        1,              // count
```

```
        MPI_INT,        // datatype
        MPI_SUM,        // operation
        0,              // root rank
        MPI_COMM_WORLD // communicator
    );

    // 5) Only the root process (rank 0) sees the correct global_sum.
    if (world_rank == 0) {
        printf("Hello from %d processes! Sum of ranks = %d\n",
               world_size, global_sum);
    }

    // 6) Clean up the MPI environment.
    //    No MPI functions may be called after this.
    MPI_Finalize();

    return 0;
}
```

MPI_Allreduce

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // 1) Initialize the MPI environment.
    //    Must be called before any other MPI function.
    MPI_Init(&argc, &argv);

    // 2) Find out how many processes are participating.
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // 3) Determine the rank (ID) of this process.
```

```
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

// 4) Each process sets its local value to its own rank.
int local_value = world_rank;

// 5) Prepare a variable to hold the reduced (summed) result.
//    After MPI_Allreduce, every process will have this value.
int global_sum = 0;

// 6) Perform an “all-reduce” across all ranks.
//    - sendbuf: address of this process’s data         (&local_value)
//    - recvbuf: address where result will be stored     (&global_sum)
//    - count:   number of elements to reduce            (1)
//    - datatype:MPI_INT for C `int`
//    - op:      MPI_SUM to sum all local_value entries
//    - comm:    communicator (MPI_COMM_WORLD)
//
// Unlike MPI_Reduce, MPI_Allreduce does the reduction and then
// broadcasts the result back to *all* processes, so each rank
// ends up with the final sum.
MPI_Allreduce(
   &local_value,    // sendbuf
   &global_sum,     // recvbuf
   1,               // count
   MPI_INT,         // datatype
   MPI_SUM,         // operation
   MPI_COMM_WORLD   // communicator
);

// 7) Now *every* process can use the global sum.
//    Let’s print it from each rank.
printf("Process %d of %d: global sum of ranks = %d\n",
       world_rank, world_size, global_sum);

// 8) Finalize the MPI environment.
```

```
    //    No MPI calls are allowed after this.
    MPI_Finalize();

    return 0;
}
```

MPI_Bcast

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char** argv) {
    // 1) Initialize the MPI environment.
    //    Must be called before any other MPI functions.
    MPI_Init(&argc, &argv);

    // 2) Determine the total number of processes in MPI_COMM_WORLD.
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // 3) Determine the rank (ID) of this process.
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // 4) Prepare a buffer for the message.
    //    Note: all processes must allocate the buffer, even if only root writes into it.
    char message[32];

    if (world_rank == 0) {
        // 5) Root process (rank 0) initializes the message.
        strncpy(message, "Hello from MPI_Bcast!", sizeof(message));
        // Ensure null-termination in case of overflow
```

```
        message[sizeof(message)-1] = '\0';
    }

    // 6) Broadcast the message from root to all processes.
    //    - buffer: address of the data to broadcast (&message)
    //    - count:  maximum number of elements (32 chars)
    //    - datatype: MPI_CHAR (each element is a char)
    //    - root:   rank of broadcaster (0)
    //    - comm:   communicator (MPI_COMM_WORLD)
    //
    // After this call, 'message' on every rank contains the root's string.
    MPI_Bcast(
        message,        // buffer
        32,             // count
        MPI_CHAR,       // datatype
        0,              // root rank
        MPI_COMM_WORLD  // communicator
    );

    // 7) All processes can now safely use 'message'.
    printf("Process %d/%d received message: '%s'\n",
           world_rank, world_size, message);

    // 8) Finalize the MPI environment.
    //    No MPI calls may follow this.
    MPI_Finalize();

    return 0;
}
```

MPI_Scatter

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    // 1) Initialize the MPI environment.
    MPI_Init(&argc, &argv);

    // 2) Determine total number of processes.
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // 3) Determine the rank (ID) of this process.
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // 4) Define how many elements each process should receive.
    //    For this example, we'll give each process 2 integers.
    const int elems_per_proc = 2;

    // 5) Allocate space for each process's receive buffer.
    int* recv_buffer = malloc(sizeof(int) * elems_per_proc);

    // 6) On the root process (rank 0), prepare the full data array.
    int* send_buffer = NULL;
    if (world_rank == 0) {
        // Allocate and fill world_size * elems_per_proc values
        send_buffer = malloc(sizeof(int) * world_size * elems_per_proc);
        for (int i = 0; i < world_size * elems_per_proc; i++) {
            send_buffer[i] = i;  // e.g., [0,1,2,3,4,5,...]
        }
        printf("Root process prepared data:");
        for (int i = 0; i < world_size * elems_per_proc; i++) {
            printf(" %d", send_buffer[i]);
        }
```

```
    printf("\n");
}

// 7) Scatter the data:
//    - send_buffer: array on root holding all data (ignored on non-root)
//    - elems_per_proc: number of ints each process receives
//    - MPI_INT: datatype of each element
//    - recv_buffer: where each process stores its chunk
//    - elems_per_proc: same count for receiving
//    - MPI_INT: datatype of receive buffer
//    - root = 0: the process that holds send_buffer
//    - MPI_COMM_WORLD: communicator
MPI_Scatter(
    send_buffer,          // sendbuf (root only)
    elems_per_proc,       // sendcount
    MPI_INT,              // sendtype
    recv_buffer,          // recvbuf (all ranks)
    elems_per_proc,       // recvcount
    MPI_INT,              // recvtype
    0,                    // root rank
    MPI_COMM_WORLD        // communicator
);

// 8) Each process now prints what it received.
printf("Process %d received:", world_rank);
for (int i = 0; i < elems_per_proc; i++) {
    printf(" %d", recv_buffer[i]);
}
printf("\n");

// 9) Clean up root's send_buffer and all ranks' recv_buffer.
if (world_rank == 0) free(send_buffer);
free(recv_buffer);

// 10) Finalize the MPI environment.
MPI_Finalize();
```

```
    return 0;
}
```

MPI_Scatterv

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    // 1) Initialize the MPI environment.
    MPI_Init(&argc, &argv);

    // 2) Find out how many processes are participating.
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // 3) Find out this process's rank.
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // 4) Decide, for demonstration, that process i should receive (i+1) integers.
    //    Total elements = 1 + 2 + ... + world_size = world_size*(world_size+1)/2
    int *sendcounts = NULL;
    int *displs     = NULL;
    int total_elems = world_size * (world_size + 1) / 2;

    // 5) Root (rank 0) allocates and initializes the full send buffer,
    //    plus the arrays describing counts & displacements.
    int *sendbuf = NULL;
    if (world_rank == 0) {
        sendbuf = malloc(sizeof(int) * total_elems);
        sendcounts = malloc(sizeof(int) * world_size);
        displs     = malloc(sizeof(int) * world_size);
```

```
    // Fill sendcounts: process i gets (i+1) elements
    for (int i = 0; i < world_size; i++) {
        sendcounts[i] = i + 1;
    }

    // Compute displacements: start of each block in sendbuf
    displs[0] = 0;
    for (int i = 1; i < world_size; i++) {
        displs[i] = displs[i-1] + sendcounts[i-1];
    }

    // Initialize the big array with sequential values
    for (int i = 0; i < total_elems; i++) {
        sendbuf[i] = i;
    }

    // (Optional) print what root is scattering
    printf("Root will scatter array of %d elements:\n", total_elems);
    for (int i = 0; i < total_elems; i++) {
        printf(" %d", sendbuf[i]);
    }
    printf("\nSendcounts:");
    for (int i = 0; i < world_size; i++) {
        printf(" %d", sendcounts[i]);
    }
    printf("\nDispls:");
    for (int i = 0; i < world_size; i++) {
        printf(" %d", displs[i]);
    }
    printf("\n\n");
}

// 6) Each process allocates a receive buffer sized to its intended count.
int recvcount;
// We need to inform non-root ranks of how many they'll receive.
```

```
// One simple way: broadcast sendcounts array first.
if (world_rank == 0) {
   recvcount = sendcounts[0];  // placeholder for consistency
}
// Ensure everyone knows sendcounts (so they know their recvcount)
if (world_rank == 0) {
   // Root broadcasts the entire sendcounts array
   MPI_Bcast(sendcounts, world_size, MPI_INT, 0, MPI_COMM_WORLD);
} else {
   sendcounts = malloc(sizeof(int) * world_size);
   MPI_Bcast(sendcounts, world_size, MPI_INT, 0, MPI_COMM_WORLD);
}
recvcount = sendcounts[world_rank];

int *recvbuf = malloc(sizeof(int) * recvcount);

// 7) Perform the variable-length scatter:
//    - sendbuf:    full array at root (ignored elsewhere)
//    - sendcounts: elements sent to each rank
//    - displs:     offsets where each block begins in sendbuf
//    - MPI_INT:    datatype of sendbuf elements
//    - recvbuf:    local buffer for each process
//    - recvcount:  number of elements this process receives
//    - MPI_INT:    datatype for recvbuf
//    - root = 0:   the scattering process
//    - MPI_COMM_WORLD: communicator
MPI_Scatterv(
   sendbuf,      // send buffer (root only)
   sendcounts,   // send counts per rank
   displs,       // displacements per rank
   MPI_INT,      // send datatype
   recvbuf,      // receive buffer (all ranks)
   recvcount,    // receive count
   MPI_INT,      // receive datatype
   0,            // root rank
   MPI_COMM_WORLD
```

```
    );

    // 8) Each process prints its received chunk.
    printf("Process %d received %d elements:", world_rank, recvcount);
    for (int i = 0; i < recvcount; i++) {
        printf(" %d", recvbuf[i]);
    }
    printf("\n");

    // 9) Cleanup
    free(recvbuf);
    if (world_rank == 0) {
        free(sendbuf);
        free(sendcounts);
        free(displs);
    } else {
        free(sendcounts);
    }

    // 10) Finalize MPI.
    MPI_Finalize();
    return 0;
}
```

MPI_Gather

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    // 1) Initialize MPI environment.
```

```
MPI_Init(&argc, &argv);

// 2) Determine total number of processes.
int world_size;
MPI_Comm_size(MPI_COMM_WORLD, &world_size);

// 3) Determine this process's rank.
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

// 4) Each process prepares its local value (here, just its rank).
int local_value = world_rank;

// 5) On the root, allocate a buffer large enough to hold every rank's data.
int *gathered = NULL;
if (world_rank == 0) {
   gathered = malloc(sizeof(int) * world_size);
}

// 6) Gather all local_values to the root process:
//    - sendbuf: address of this process's data (&local_value)
//    - sendcount: number of items sent by each process (1)
//    - sendtype: MPI_INT
//    - recvbuf: address where root collects data (gathered)
//               (ignored on non-root ranks)
//    - recvcount: number of items root expects from each process (1)
//    - recvtype: MPI_INT
//    - root: 0
//    - comm: MPI_COMM_WORLD
MPI_Gather(
   &local_value,    // send buffer (each rank)
   1,               // send count
   MPI_INT,         // send datatype
   gathered,        // receive buffer (root only)
   1,               // receive count per rank
   MPI_INT,         // receive datatype
```

```
        0,              // root rank
        MPI_COMM_WORLD   // communicator
    );

    // 7) Root prints the array of gathered ranks.
    if (world_rank == 0) {
        printf("Root has gathered ranks:");
        for (int i = 0; i < world_size; i++) {
            printf(" %d", gathered[i]);
        }
        printf("\n");
        free(gathered);
    }

    // 8) Finalize the MPI environment.
    MPI_Finalize();
    return 0;
}
```

MPI_Allgather

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    // 1) Initialize the MPI environment.
    MPI_Init(&argc, &argv);

    // 2) Determine total number of processes in MPI_COMM_WORLD.
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
```

```
// 3) Determine the rank (ID) of this process.
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

// 4) Each process prepares its local value.
//    Here we simply use the rank as the value.
int local_value = world_rank;

// 5) Allocate a receive buffer on each process to hold
//    data from all processes: world_size elements.
int *recvbuf = malloc(sizeof(int) * world_size);
if (recvbuf == NULL) {
    fprintf(stderr, "Process %d: failed to allocate recvbuf\n", world_rank);
    MPI_Abort(MPI_COMM_WORLD, 1);
}

// 6) Perform the all-gather:
//    - sendbuf: address of this process's data (&local_value)
//    - sendcount: number of items sent by this process (1)
//    - sendtype: MPI_INT
//    - recvbuf: array where each process stores the concatenated results
//    - recvcount: number of items contributed by each process (1)
//    - recvtype: MPI_INT
//    - comm: MPI_COMM_WORLD
//
// After this call, recvbuf on every process contains:
//   [ value_from_rank_0, value_from_rank_1, ..., value_from_rank_{N-1} ]
MPI_Allgather(
    &local_value,    // send buffer
    1,               // send count
    MPI_INT,         // send datatype
    recvbuf,         // receive buffer
    1,               // receive count per process
    MPI_INT,         // receive datatype
    MPI_COMM_WORLD   // communicator
);
```

```
    // 7) Print the gathered array from each process's perspective.
    printf("Process %d received array:", world_rank);
    for (int i = 0; i < world_size; i++) {
        printf(" %d", recvbuf[i]);
    }
    printf("\n");

    // 8) Clean up.
    free(recvbuf);

    // 9) Finalize the MPI environment.
    MPI_Finalize();
    return 0;
}
```